

Team Run Members:

Daniel Herrera (Team Leader)

Harut Kulakchian

Anthony Jimenez

Osbaldo Bravo

Carlos Martine

Team Run Architecture Design

Mobile App

Front End:

- **HTML5** – Used to build the structure of the application interface.
- **Tailwind CSS** – Used for styling and layout of the user interface.
- **JavaScript** – Handles user interactions and communication with the backend through API requests.

Back End:

- **Node.js** – runtime environment used to run the backend server
- **Express.js** – framework used to build REST API endpoints and manage server routing
- **JWT (JSON Web Token)** – used for authentication and session security
- **bcrypt** – encrypts user passwords before storing them

Database:

- **MongoDB Atlas** – cloud based NoSQL database used to store user data, workouts, hydration logs, and character progression
- **Mongoose** – library used to define schemas and interact with MongoDB

API's:

- **HealthKit / Google Fit (Optional pick)** – These APIs may be used in the future to retrieve workout or activity data from wearable devices such as smartwatches or fitness trackers. This would allow workouts to automatically sync with the application instead of requiring manual entry.
- **OpenAI API** – Planned for potential features such as AI-generated workout suggestions, adaptive training plans, or interactive RPG dialogue for enemies or characters.

Package/Build Manager

- **npm (Node Package Manager)** – Used to install and manage project dependencies such as Express, JWT, bcrypt, and Mongoose.
- **Node.js Runtime** – Runs the backend server and processes API requests from the frontend.
- **Git / GitHub** – Used for version control and collaboration between team members.

Task Assignments:

→Frontend development and UI components -

Anthony Jimenez

→Backend development and API logic -

Carlos Martinez

→AI features and character progression logic -

Osbaldo Bravo

→Database schema design and database queries -

Harut Kulakchian

→Maps integration and external APIs -

Daniel Herrera

URL

- [Project Requirement Document \(PRD\)](#)
- [Team Run Metrics](#)
- [GitHub](#)

For a Mobile App:

Database Tables (Models):

User — account information, workout streak, level, experience points.

Workout — exercise type, body part trained, reps, sets, weight, time, and date logged.

WaterLog — amount of water consumed (ml) and timestamp.

Character — player character stats such as strength, stamina, and experience.

Boss/Raid — required stats to unlock, difficulty level, and rewards.

Gear / UserGear — cosmetic or stat gear unlocked and equipped by the user.

ReminderSetting — hydration or workout reminder preferences.

Views:

Dashboard (My Week) — shows weekly workouts, hydration progress, and character growth.

Log Workout — user selects body part, exercise type, reps/weight, and logs workout activity.

Water Tracker — quick water logging buttons with a hydration progress bar.

Warm-Up — preset or custom warm-up routines with timer support.

Raids/Boss — unlocked areas where users can challenge bosses based on their stats.

Gear/Wardrobe — unlock and equip character gear and cosmetic items.

Rest Mode — appears after several missed days and allows the user to resume progress.

Settings — user permissions, notification settings, and optional device integrations.

URLs:

/login

/log/workout

/log/water

/warmup

/week

/raids

/gear

/rest

/settings

/api/workouts

/api/water

/api/summary/week

/api/player

/api/leaderboard

Design Considerations:

- **Decoupling** – The system is separated into different layers such as controllers, services, and database models. This makes the code easier to

maintain and allows different parts of the system to change without breaking the entire application.

- **Anti-Cheat** – Workout progress and character stats will be calculated on the server side instead of the client side. This helps prevent users from manually modifying data or gaining unfair advantages.
- **Streak / Rest System** – The application tracks workout streaks. If a user misses multiple days of activity, the system will enter a rest mode and provide an option to resume activity later.
- **Accessibility / User Experience** – The interface will focus on simple interactions such as quick workout logging, clear progress feedback, and visual animations that reflect the players actions and progress.

For a Web App:

Deployment:

The application will be deployed as a web based platform. The frontend will run in the browser while the backend server will handle API requests, authentication, and database communication.

We plan to use a CI/CD workflow so that code changes pushed to the repository are automatically tested and deployed. This helps ensure that updates can be delivered quickly while maintaining stability.

For hosting, the team plans to use Railway. Railway provides an easy way to deploy full stack applications and connect them with cloud services such as MongoDB Atlas.

Development/Deployment Environments:

Development will take place locally on each team members machine using Node.js and npm. Version control will be managed through Git and GitHub so all team members can work on the project.

The backend server will run using Node.js and Express while the database will be hosted on MongoDB Atlas. This allows the team to access the same cloud database during development and testing.

For deployment, the application will run on Railway while continuing to connect to MongoDB Atlas for better data storage.

- **URLs:**

- Application Design
- [Project requirement](#)
- [Wiki / GitHub](#)

REST API:

- The application will use REST API endpoints to allow the frontend to communicate with the backend server.
- POST /auth/register
 - Parameters: { username, email, password }
 - Description: Creates a new user account and stores the encrypted password in the database.
- POST /auth/login
 - Parameters: { email, password }
 - Description: Authenticates a user and returns a secure login token.
- GET /player
 - Parameters: None
 - Description: Retrieves the player's profile information including level, experience, and workout progress.
- POST /workout
 - Parameters: { exercise, bodyPart, reps, sets, weight, duration }
 - Description: Saves workout data to the database and updates the player's character stats and experience points.

- POST /water
- Parameters: { amountML }
- Description: Logs water intake for the user and updates hydration progress.
- GET /leaderboard
- Parameters: None
- Description: Retrieves the top users based on level, experience, or completed workouts.

- **Views:** (IWIP) = Image is a work in progress

Dashboard (“My Week”) (IWIP)— weekly workouts/water/boss fights.

[Log in screen](#) — the first thing users will see when opening the application

[Log Workout](#) — choose body part, exercise, reps/weight, plays animation.

[Water Tracker](#) — quick amounts + progress bar + buffs.

[Warm-Up](#) — preset or custom with timer/video + avatar.

Raids/Boss (IWIP) — zone unlocks, challenge boss, rewards.

[Rewards](#) — have your character grow stronger and unlock/equip gear.

Rest Mode (IWIP)— appears after 3 missed days, resume button.

Settings (IWIP)— permissions, HR device link

Users Example:

User_id	uuid	PK	User identifier
email	text	UNIQUE, NOT NULL	login email
password	text	NOT NULL	Password
created_at	Time stamp	DEFAULT(now)	Account creation

```
CREATE TABLE users (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  email TEXT UNIQUE NOT NULL,
```

```
password_hash TEXT NOT NULL,  
created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()  
);
```

Workout example:

user_id	uuid	FK, NOT NULL	User id
type	text	NOT NULL	Name of workout
reps	int	NOT NULL	# of reps
sets	int	NOT NULL	# of sets

```
CREATE TABLE workouts (  
  id BIGSERIAL PRIMARY KEY,  
  user_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,  
  type TEXT NOT NULL,  
  reps INT CHECK (reps IS NULL OR reps >= 0),  
  sets INT CHECK (sets IS NULL OR sets >= 0),  
);
```
